Sequential Testing for A/B Experiments

A Focused Guide in Three Acts

Introduction

The problem. Standard hypothesis tests guarantee a controlled false positive rate only when results are analysed once, at a pre-specified sample size. In practice, teams routinely monitor experiments and peek at interim results — inflating the Type I error rate well beyond the nominal α.

The solution. *Sequential testing* provides statistical methods whose error guarantees hold regardless of the number or timing of analyses. The confidence intervals become wider, but it can still reduce overall runtime if effects become clear at an early stage.

This guide. Three acts, each with an interactive simulation, technical exposition, and key formulas:

1. **The Peeking Problem** — Why interim analyses invalidate standard tests.
2. **The Eppo Solution**— Confidence sequences, variance reduction, and the recommended **hybrid approach**.
3. **Alternative Methods**— Bonferroni, Pocock, and O'Brien–Fleming corrections for teams without a sequential testing platform.

Target audience. Data scientists and analysts familiar with A/B testing, hypothesis testing, and confidence intervals.

Act 1 — The Peeking Problem

Simulation

Below is a simulation of an A/B test. Specify the effect size, number of users, and significance level. The plot shows the running difference in means and the standard 95% confidence interval.

Simulation 1: fixed-horizon confidence intervals.

Relative effect size (%): 0%

Control conversion rate (10.0%)

Number of users (n = 10000)

α (0.05)

Default: α = 0.05

Relative effect size (%) (0%)

Default: effect size = 0%

Number of peeks (K) (14)

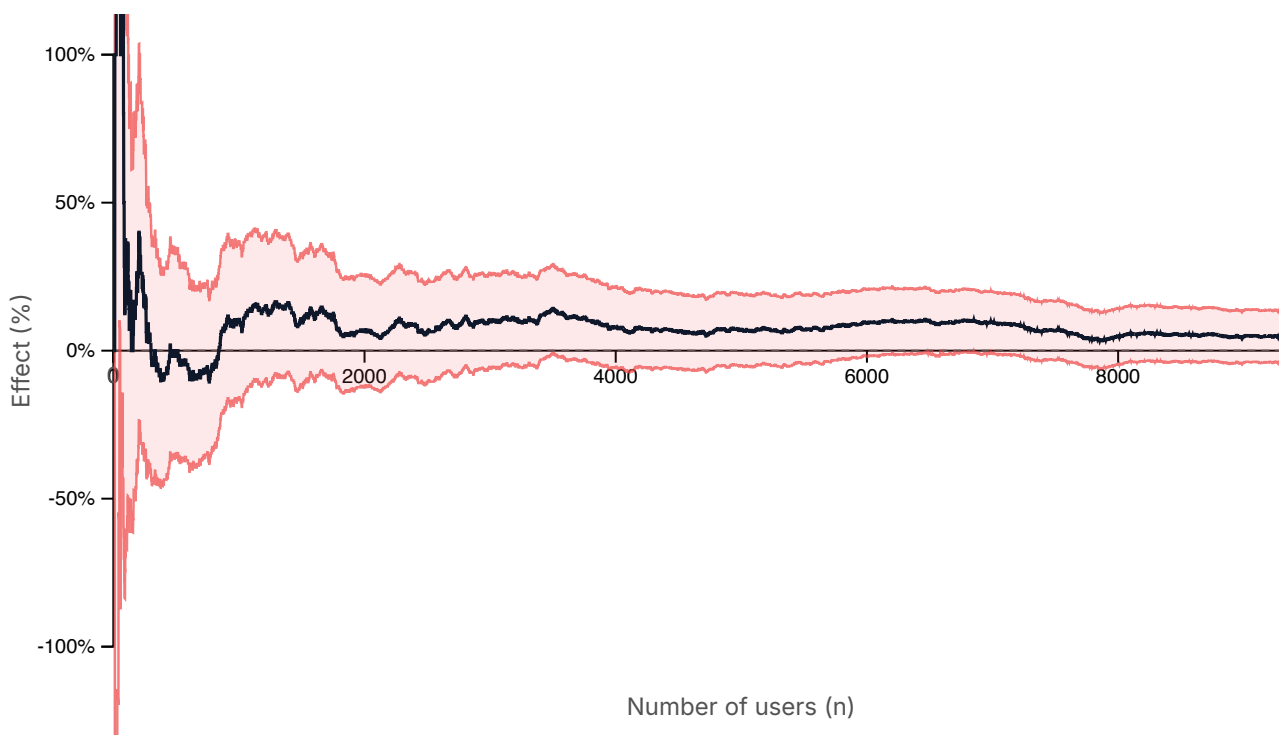
Estimated Power (1 - β)

Inapplicable since there is no true effect

Computed from effect size, n, and α (two-sided z-test approximation). Shown as inapplicable when effect size is 0.

Re-randomize

Standard 95% CI



In this simulation, would peeking 14 times at equal time intervals during the test show at least one statistically significant result?

No

Share of simulations in which peeking 14 times at equal time intervals during the test would show at least one statistically significant result, across 1000 repetitions:

Run 1000 repetitions

22.5%

Simulation assumptions and notes

Assumptions

- Binary outcome metric (Bernoulli), modeled as conversion in each arm.
- Uplift is modeled as relative lift on baseline: $\text{uplift} = (\bar{p}_B - \bar{p}_A)/\bar{p}_A$, with generator $p_B = p_A(1 + \text{lift})$.
- Null hypothesis is $H_0 : \text{lift} = 0$ (equivalently, no treatment effect).
- Standard deviation follows Bernoulli variance in each arm: $\sigma_A = \sqrt{p_A(1 - p_A)}$, $\sigma_B = \sqrt{p_B(1 - p_B)}$, so $\text{SE}(\hat{p}_B - \hat{p}_A) = \sqrt{(\sigma_A^2 + \sigma_B^2)/n}$.
- Equal traffic split: 50% control and 50% treatment.
- Control conversion baseline is user-specified via slider.
- Independent users/events within and across arms (no clustering or interference).
- No missing data, no delayed outcomes, and no sample-ratio mismatch.
- Two-sided significance check at each look: CI crossing zero is treated as significant.
- Peeks occur at exactly K equal time intervals over the test duration.

Notes

- The Monte Carlo estimate uses 1000 repetitions, so displayed rates include simulation noise.
- Example: baseline 10% with +4% relative lift gives treatment 10.4%.
- Estimated power is shown as $\approx 100\%$ when extremely close to 1, rather than exactly 100%.

Why does this happen?

Standard confidence intervals only control the false positive rate if you look once a

pre-specified sample size. Each additional peek gives another chance to get a "significant" result by luck.

How bad is it?

Values below are calibrated to the default simulation settings ($\alpha = 0.05$, $n = 10000$) using equal-interval peeks.

Checking schedule	Actual false positive rate
Once at the end (as designed)	$\sim 5\%$
Daily for 1 week	$\sim 17\%$
Daily for 2 weeks	$\sim 22\%$
Daily for 4 weeks (28 looks)	$\sim 29\%$
Continuously (every observation)	Can exceed 60%

With daily checks over a 2-week experiment, about one in five "significant" results will be a false positive. Features released on this basis may have no real effect.

Key Takeaway

Sequential methods control the probability of ever making a false positive, no matter when we stop. They use stricter or time-dependent thresholds to account for repeated checking. This way, we can monitor results as often as we like, and the error rate stays controlled.

Why peeking inflates Type I error

In a fixed-horizon A/B test (one planned analysis), we test:

$$H_0 : \tau = 0, \qquad \hat{\tau}_n = \bar{X}_{B,n} - \bar{X}_{A,n}$$

With a standard 95% CI,

$$\hat{\tau}_n \pm 1.96\,\widehat{\mathrm{SE}}_n$$

the false positive rate is controlled at one pre-specified look:

$$\Pr(\text{reject } H_0 \text{ at one look}) = \alpha = 0.05$$

If we repeatedly check the data, we effectively run many tests. The probability of at least one false positive becomes:

$$\Pr(\text{at least one false positive in } K \text{ looks})$$

Under independence this is:

$$1 - (1-\alpha)^K$$

and a useful approximation for small α is:

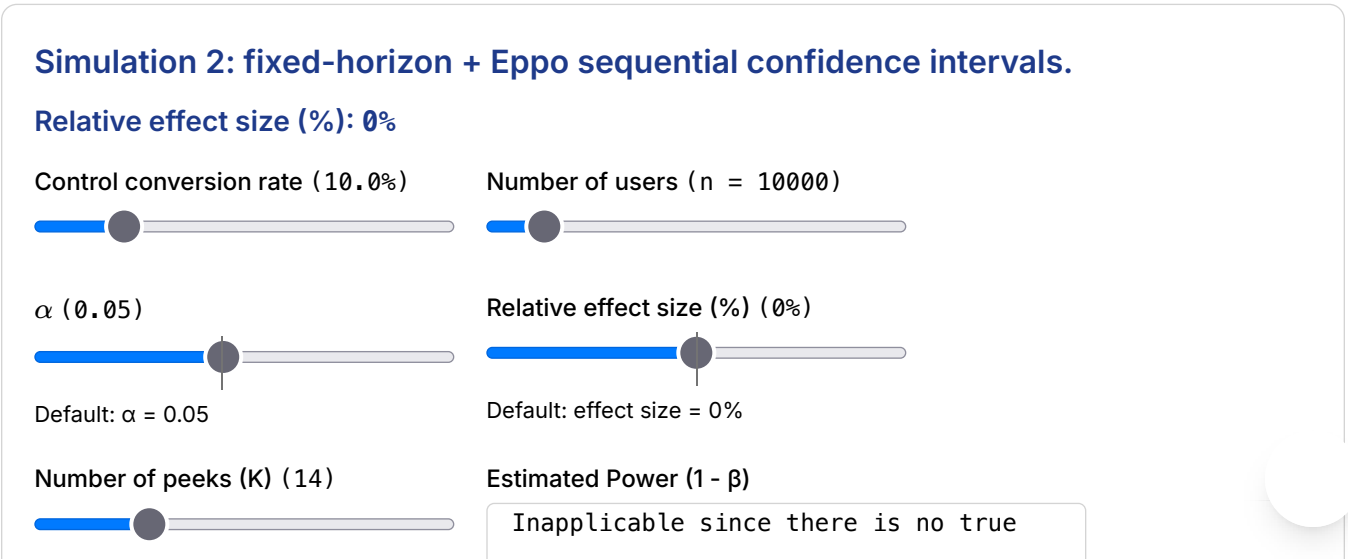
$$1 - (1-\alpha)^K \approx K\alpha$$

Real interim looks are positively correlated, so the exact value is lower than the independence formula, but still substantially above α. That is why repeated peeking inflates Type I error.

Act 2 — The Eppo Solution

Simulation

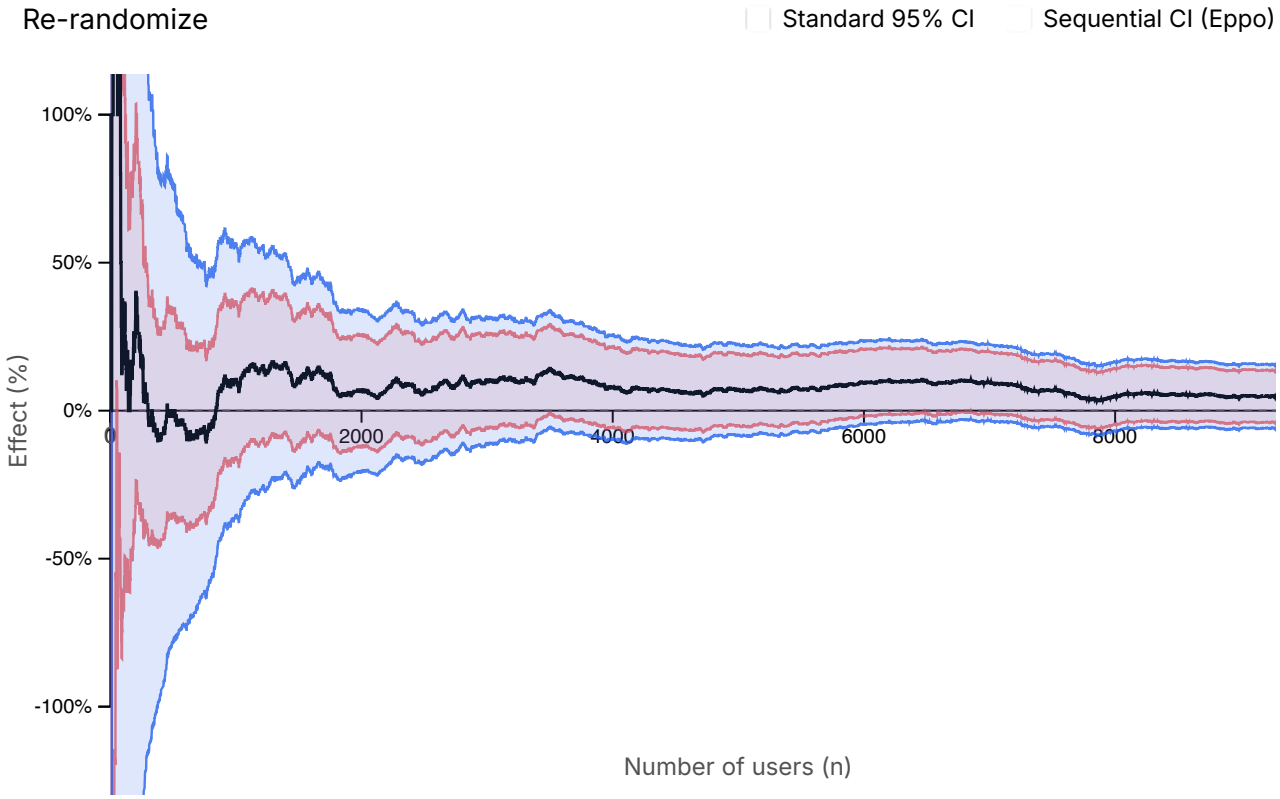
This is the same simulation as Act 1, plus Eppo's sequential confidence interval (blue) on top of the standard fixed-horizon confidence interval (red). The blue interval is typically wider at any single look, but it keeps error control valid under repeated monitoring.



effect

Computed from effect size, n, and α (two-sided z-test approximation). Shown as inapplicable when effect size is 0.

Re-randomize



In this simulation, would peeking 14 times at equal time intervals during the test show at least one statistically significant result?
No

Share of simulations in which peeking 14 times at equal time intervals during the test would show at least one statistically significant result, across 1000 repetitions:

Run 1000 repetitions

Method	Share crossing
Standard 95% CI	22.5%
Sequential CI (Eppo)	5.9%

Simulation assumptions and notes

Assumptions

- Binary outcome metric (Bernoulli), modeled as conversion in each arm.
- Uplift is modeled as relative lift on baseline: $\text{uplift} = (\bar{p}_B - \bar{p}_A)/\bar{p}_A$, with generator $p_B = p_A(1 + \text{lift})$.
- Null hypothesis is $H_0 : \text{lift} = 0$ (equivalently, no treatment effect).
- Standard deviation follows Bernoulli variance in each arm: $\sigma_A = \sqrt{p_A(1 - p_A)},\ \sigma_B = \sqrt{p_B(1 - p_B)}$, so $\text{SE}(\hat{p}_B - \hat{p}_A) = \sqrt{(\sigma_A^2 + \sigma_B^2)/n}$.
- Equal traffic split: 50% control and 50% treatment.
- Control conversion baseline is user-specified via slider.
- Independent users/events within and across arms (no clustering or interference).
- No missing data, no delayed outcomes, and no sample-ratio mismatch.
- Two-sided significance check at each look: CI crossing zero is treated as significant.
- Peeks occur at exactly K equal time intervals over the test duration.

Notes

- The Monte Carlo estimate uses 1000 repetitions, so displayed rates include simulation noise.
- Example: baseline 10% with +4% relative lift gives treatment 10.4%.
- Estimated power is shown as $\approx 100\%$ when extremely close to 1, rather than exactly 100%.

Why does this work?

The fixed-horizon CI controls error for one planned analysis. Eppo's sequential CI uses a time-dependent boundary, so the guarantee is valid no matter when or how often you peek.

Tradeoff: at any single look, the interval is wider; benefit: no hidden inflation from repeated checks.

How different is it?

Values below are calibrated to default settings ($\alpha = 0.05$, $n = 10000$, baseline 10%, no true effect) using equal-interval peeks.

Checking schedule	Standard 95% CI	Sequential CI (Eppo)
Once at the end (1 look)	$\sim 5\%$	$\sim 2\%$

Checking schedule	Standard 95% CI	Sequential CI (Eppo)
Daily for 1 week (7 looks)	$\sim 16\%$	$\sim 4\%$
Daily for 2 weeks (14 looks)	$\sim 22\%$	$\sim 5\%$
Daily for 4 weeks (28 looks)	$\sim 28\%$	$\sim 5\%$
Very frequent checks (50 looks)	$\sim 33\%$	$\sim 6\%$

Under repeated peeking, the standard CI inflates false positives substantially, while the sequential CI stays close to the target error level.

Key Takeaway

Sequential intervals trade narrowness at a single look for validity across all looks. If your team monitors live, this is the right tradeoff.

The Hybrid Approach

The sequential CI is always wider than a fixed-horizon CI at any given sample size. For the **primary KPI** — the metric the experiment is designed to move — this width penalty reduces statistical power.

Apply sequential testing where early stopping adds the most value (safety guardrails), and fixed-horizon testing where statistical power matters most (the primary metric).

The hybrid approach partitions metrics into two categories:

- **Guardrail KPIs** (revenue, error rate, latency, etc.): monitored continuously with a **sequential CI**. If the CI excludes zero on the harmful side, abort the experiment immediately.
- **Primary KPI** (the metric the experiment targets): analysed with a **standard fixed-horizon CI** at the pre-planned end date. No peeking penalty. Full statistical pow

What you gain

	Full sequential	Hybrid
Primary KPI CI width	Wider (~10–40%)	Standard (no penalty)
Primary KPI power	Reduced	Full
Guardrail protection	Continuous	Continuous
Early stopping for success	Yes	No
Early stopping for harm	Yes	Yes
Complexity	Higher	Lower

Key Takeaway

The hybrid approach in one sentence: Monitor guardrail KPIs with a sequential CI for early abort; analyse the primary KPI with a standard CI at the planned end-date.

This gives you the safety of sequential testing where it matters most (preventing harm) without paying the power penalty on the metric you care about most (the primary KPI).

This is the recommended approach for most A/B testing programmes.

Mathematical Formulation

In this simulation, the displayed effect and the CI boundaries are both expressed as percentage uplift:

$$\hat{u}_n = 100 \cdot \frac{\bar{X}_{B,n} - \bar{X}_{A,n}}{\bar{X}_{A,n}}$$

The CI half-width is computed from the running standard error $\widehat{\mathrm{SE}}_n$ of the difference in means, then divided by the control mean to convert to percentage scale. A fixed-horizon 95% CI uses

$$\hat{u}_n \pm 100 \cdot \frac{1.96\,\widehat{\mathrm{SE}}_n}{\bar{X}_{A,n}}$$

and is valid at one pre-specified analysis.

A sequential CI replaces 1.96 with a time-dependent multiplier:

$$\hat{u}_n \pm 100 \cdot \frac{\widehat{\mathrm{SE}}_n}{\bar{X}_{A,n}} \sqrt{\frac{n+\nu}{n} \log\left(\frac{n+\nu}{\nu\alpha}\right)}$$

where ν is a tuning parameter and α is the target Type I error level.

This yields time-uniform coverage:

$$\Pr(\tau \in \mathrm{CI}_n \text{ for all } n \geq 1) \geq 1 - \alpha$$

So unlike fixed-horizon CIs, the guarantee still holds under continuous monitoring.

Act 3 — Alternative Methods

Three group sequential methods for controlling false positives under interim analyses, for teams implementing sequential monitoring without a dedicated platform.

How these methods differ

Suppose you plan to analyse your experiment at K pre-specified interim time points. Using the standard $\alpha = 0.05$ threshold at each analysis inflates the overall Type I error rate (Act 1).

The fix: **adjust the per-analysis significance level** so that the family-wise error rate remains at α. The three methods below differ in *how* they allocate the error budget across the K analyses.

- **Bonferroni:** equal allocation, α/K per analysis. Conservative because it ignores correlation across analyses.
- **Pocock:** constant critical value calibrated to the joint distribution of the test statistics across analyses. Tighter than Bonferroni.
- **O'Brien–Fleming:** front-loaded allocation. Very strict early, nearly standard at the final analysis.

Eppo's approach (Act 2) does not require pre-specifying K— it provides a continuously valid guarantee.

Simulation

This extends the Act 1/2 simulation by adding Bonferroni, Pocock, and O'Brien–Flemin

confidence intervals, so you can compare all methods under the same settings.

Method 1: Bonferroni

The simplest correction. Divide α by K. One line of code.

Method 1: Bonferroni Correction (Simplest)

The idea

Before the experiment starts, fix the number of planned analyses K (e.g. weekly for 4 weeks: $K = 4$). At each analysis, use a significance level of α/K instead of α.

The recipe

1. Choose K (number of planned peeks) and α (e.g. 0.05).
2. At each peek $k = 1, \ldots, K$, compute the standard CI using the adjusted significance level:

$$\mathrm{CI}_k = \hat{\tau}(t_k) \ \pm \ \hat{\sigma}_{\hat{\tau}}(t_k) \cdot z_{\alpha/(2K)}$$

where $\hat{\tau}(t_k)$ is the estimated treatment effect at peek k, $\hat{\sigma}_{\hat{\tau}}(t_k)$ is its standard error, and $z_{\alpha/(2K)}$ is the critical value from a Normal distribution at the adjusted significance level.

3. If the CI does not cross 0: stop and declare significance.
4. If $k = K$ and the CI crosses 0: no significant effect.

Critical Values

K (peeks)	α/K	$z_{\alpha/(2K)}$	CI multiplier vs. 1.96
1	0.050	1.96	1.00× (no correction)
2	0.025	2.24	1.14×
4	0.0125	2.50	1.28×
8	0.00625	2.73	1.39×
12	0.00417	2.87	1.46×
20	0.0025	3.02	1.54×

K (peeks)	α/K	$z_{\alpha/(2K)}$	CI multiplier vs. 1.96
52	0.00096	3.29	1.68× (weekly for a year)

With $K = 4$, replace $z = 1.96$ with $z = 2.50$. The CI is 28% wider at each analysis, but the family-wise error rate is controlled at $alpha$.

Implementation step by step

Given data arrays `control[]` and `treatment[]` at each scheduled peek:

1. **Compute group statistics:**

$$\bar{Y}_0 = \frac{1}{n_0}\sum_{i=1}^{n_0} Y_{0,i}, \qquad s_0^2 = \frac{1}{n_0 - 1}\sum_{i=1}^{n_0}(Y_{0,i} - \bar{Y}_0)^2$$

2. **Compute the estimated treatment effect and its standard error:**

$$\hat{\tau} = \bar{Y}_1 - \bar{Y}_0, \qquad \mathrm{SE} = \sqrt{\frac{s_0^2}{n_0} + \frac{s_1^2}{n_1}}$$

3. **Look up the adjusted critical value:**

$$z^* = z_{\alpha/(2K)} = \Phi^{-1}\left(1 - \tfrac{\alpha}{2K}\right)$$

where Φ^{-1} is the inverse of the standard Normal CDF (e.g. `scipy.stats.norm.ppf` in Python).

4. **Construct the CI and decide:**

$$\mathrm{CI} = \left[\hat{\tau} - z^* \cdot \mathrm{SE},\ \hat{\tau} + z^* \cdot \mathrm{SE}\right]$$

If the CI does not contain 0, stop and declare significance. Otherwise, wait until the next scheduled peek.

Pros and cons

+ Dead simple. One line of code: change 1.96 to $z_{\alpha/(2K)}$.
+ Always valid. Works for any test statistic, any distribution.

– Conservative. The CI is wider than necessary because Bonferroni ignores the correlation between test statistics at successive peeks.
– Scales poorly. As K grows, the correction becomes increasingly harsh.

Method 2: Pocock (1977)

Constant critical boundary. Tighter than Bonferroni by exploiting correlation.

Method 2: Pocock Boundaries (1977)

The idea

Like Bonferroni, pre-specify K equally-spaced analysis times. But instead of Bonferroni's crude α/K split, use a constant critical value c_P that accounts for the correlation between test statistics at different peeks.

Because the test statistics $Z_1, Z_2, \ldots, Z_K$ at successive analyses share overlapping data, they are positively correlated. Pocock's method exploits this to derive a tighter threshold.

The recipe

1. Choose K and α.
2. Look up (or compute) the Pocock critical value $c_P(K, \alpha)$.
3. At each peek k, compute the z-statistic: $Z_k = \hat{\tau}(t_k)/\hat{\sigma}_{\hat{\tau}}(t_k)$.
4. If $|Z_k| > c_P$: stop and declare significance.

Pocock vs Bonferroni critical values

K	c_P (Pocock)	$z_{\alpha/(2K)}$ (Bonf.)	Improvement
2	2.18	2.24	3% tighter
4	2.36	2.50	6% tighter
8	2.51	2.73	8% tighter
12	2.60	2.87	9% tighter

K	c_P **(Pocock)**	$z_{\alpha/(2K)}$ **(Bonf.)**	**Improvement**
20	2.69	3.02	11% tighter

Pocock gives a constant threshold that is a few percent lower than Bonferroni's. The key advantage: the threshold is the same at every peek, so it is almost as simple to use as Bonferroni.

How the boundary is computed

Under the null, the z-statistics at successive analyses follow a multivariate Normal with correlation:

$$\mathrm{Cor}(Z_j, Z_k) = \sqrt{\frac{\min(j,k)}{\max(j,k)}} \quad \text{for } 1 \le j, k \le K$$

The Pocock boundary c_P is the unique value satisfying:

$$\mathbb{P}\Big(\max_{1 \le k \le K} |Z_k| > c_P\Big) = \alpha$$

Once c_P is known, the decision rule at each peek is identical to Bonferroni, but with c_P replacing $z_{\alpha/(2K)}$.

Method 3: O'Brien–Fleming (1979)

Decreasing threshold. Nearly no penalty at the final analysis.

Method 3: O'Brien–Fleming Boundaries (1979)

The idea

Instead of a constant threshold, use a threshold that *decreases* over time. At the first peek, demand very strong evidence; at the last peek, demand nearly the same evidence as a classical test.

The recipe

At peek k (of K total), use the critical value:

$$c_k^{\text{OBF}} \approx z_{\alpha/2} \cdot \sqrt{K/k}$$

where $z_{\alpha/2} \approx 1.96$ is the standard Normal critical value.

Critical values at each peek (K=4)

Peek k (of K=4)	c_k^{OBF}	c_P (Pocock)	$z_{\alpha/(2K)}$ (Bonf.)	$z_{\alpha/2}$ (classi
1 (25% of data)	**4.05**	2.36	2.50	1.96
2 (50% of data)	**2.86**	2.36	2.50	1.96
3 (75% of data)	**2.34**	2.36	2.50	1.96
4 (100% of data)	**2.02**	2.36	2.50	1.96

O'Brien–Fleming is extremely conservative early (threshold 4.05 at the first analysis — early stopping is rare) but barely penalises the final analysis (2.02 vs. 1.96 — only 3% wider). This makes it the most efficient method: if the experiment runs to completion, the power loss from the interim analyses is negligible.

Implementation step by step

1. **Before the experiment:** Fix K and α. At each peek k, the fraction of total data collected is $t_k = k/K$ (assuming equal spacing).

2. **At peek k, compute the critical value:**

$$c_k^{\text{OBF}} = \frac{z_{\alpha/2}}{\sqrt{t_k}} = z_{\alpha/2} \cdot \sqrt{\frac{K}{k}}$$

3. **Compute the z-statistic** (same as Bonferroni and Pocock):

$$Z_k = \frac{\hat{\tau}(t_k)}{\text{SE}(t_k)}$$

4. **Decide:** If $|Z_k| > c_k^{\mathrm{OBF}}$: reject. Otherwise: continue to peek $k+1$.

5. **Convert to a confidence interval** (optional but recommended):

$$\mathrm{CI}_k = \hat{\tau}(t_k) \;\pm\; c_k^{\mathrm{OBF}} \cdot \mathrm{SE}(t_k)$$

Note: the CI width *changes* across peeks because c_k^{OBF} varies.

Practical tips

The exact O'Brien–Fleming boundaries (computed via multivariate Normal integration) differ slightly from the $z_{\alpha/2}/\sqrt{t_k}$ approximation. The approximation is accurate for $K \leq 10$ and is what most software implements. For larger K, use a library (e.g. `statsmodels` or the R package `gsDesign`).

How methods compare in simulation

Simulation 3: fixed-horizon, Eppo, and three alternative sequential methods.

Relative effect size (%): 0%

Control conversion rate (10.0%)

Number of users (n = 10000)

α (0.05)

Default: α = 0.05

Relative effect size (%) (0%)

Default: effect size = 0%

Number of peeks (K) (6)

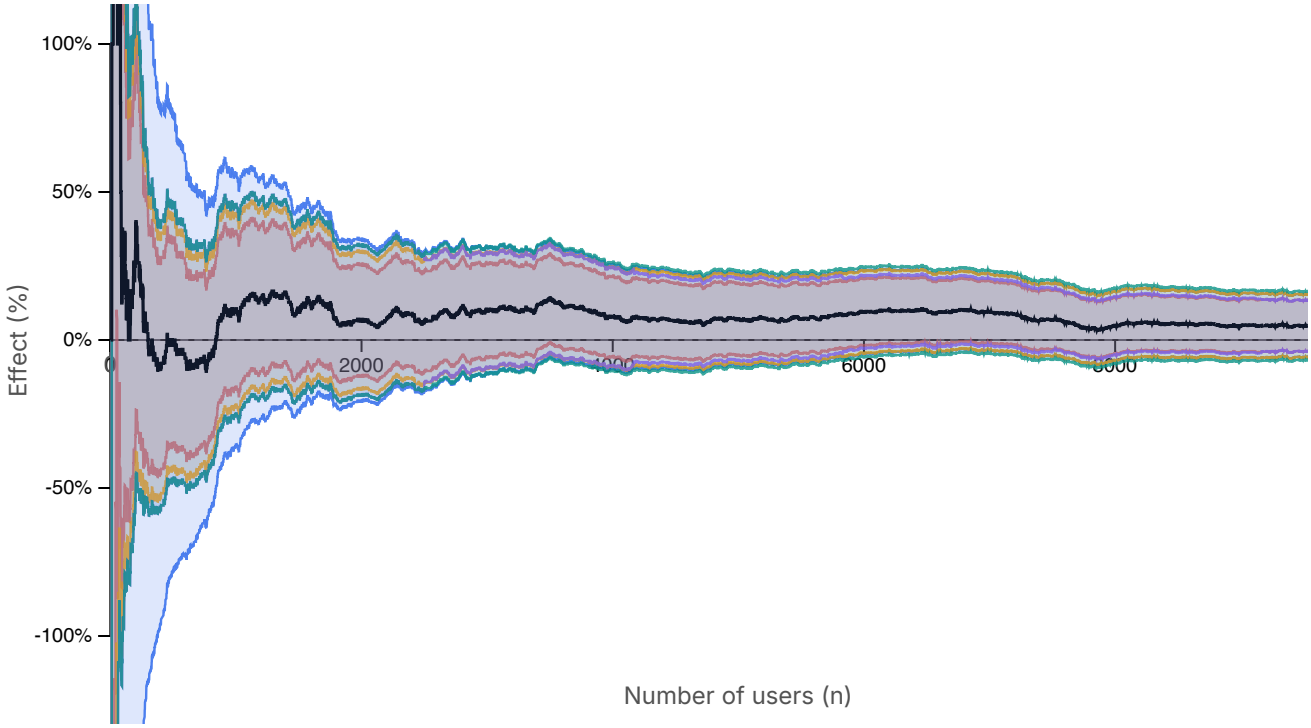
Estimated Power (1 - β)

Inapplicable since there is no true effect

Computed from effect size, n, and α (two-sided z-test approximation). Shown as inapplicable when effect size is 0.

Re-randomize

Standard 95% CI | Sequential CI (Eppo) | Pocock (K=6) | O'Brien–Fleming (K=6) | Bonferroni (K=6)



In this simulation, would peeking 6 times at equal time intervals during the test show at least one statistically significant result?
No

Result interpretation: click “Run 1000 repetitions” to estimate how often each method crosses significance under the current settings.

Bonferroni: most conservative among the three DIY methods (lowest crossing share).
Pocock: less conservative than Bonferroni with the same threshold at each look.
O'Brien–Fleming: very strict early, then close to classical thresholds at later looks.
Sequential CI (Eppo): anytime-valid and typically conservative in this setup.

Share of simulations in which peeking 6 times at equal time intervals during the test would show at least one statistically significant result, across 1000 repetitions:

Run 1000 repetitions

Method	Share crossing
Standard 95% CI	16.3%
Sequential CI (Eppo)	4.2%
Pocock (K=6)	6.3%
O'Brien–Fleming (K=6)	9.4%

Method	Share crossing
Bonferroni (K=6)	3.0%

Simulation assumptions and notes

Assumptions

- Binary outcome metric (Bernoulli), modeled as conversion in each arm.
- Uplift is modeled as relative lift on baseline: $\text{uplift} = (\bar{p}_B - \bar{p}_A)/\bar{p}_A$, with generator $p_B = p_A(1 + \text{lift})$.
- Null hypothesis is $H_0 : \text{lift} = 0$ (equivalently, no treatment effect).
- Standard deviation follows Bernoulli variance in each arm: $\sigma_A = \sqrt{p_A(1 - p_A)}$, $\sigma_B = \sqrt{p_B(1 - p_B)}$, so $\text{SE}(\hat{p}_B - \hat{p}_A) = \sqrt{(\sigma_A^2 + \sigma_B^2)/n}$.
- Equal traffic split: 50% control and 50% treatment.
- Control conversion baseline is user-specified via slider.
- Independent users/events within and across arms (no clustering or interference).
- No missing data, no delayed outcomes, and no sample-ratio mismatch.
- Two-sided significance check at each look: CI crossing zero is treated as significant.
- Peeks occur at exactly K equal time intervals over the test duration.

Notes

- The Monte Carlo estimate uses 1000 repetitions, so displayed rates include simulation noise.
- Example: baseline 10% with +4% relative lift gives treatment 10.4%.
- Estimated power is shown as $\approx 100\%$ when extremely close to 1, rather than exactly 100%.

Comparison

The CI-width rows below are illustrative values for $K = 4$, independent of the slider above.

	Bonferroni	Pocock	OBF	Eppo
Pre-specify K?	Yes	Yes	Yes	No
Threshold type	Constant	Constant	Decreasing	Continuous
CI at peek 1 (K=4)	$2.50 \times \hat{\sigma}$	$2.36 \times \hat{\sigma}$	$4.05 \times \hat{\sigma}$	$\sim 3.8 \times$

	Bonferroni	Pocock	OBF	Eppo
CI at final analysis (K=4)	$2.50 \times \hat{\sigma}$	$2.36 \times \hat{\sigma}$	$2.02 \times \hat{\sigma}$	$\sim 2.3 \times \hat{\sigma}$
Valid between peeks?	No	No	No	Yes
Variance reduction?	Manual	Manual	Manual	Built-in
Implementation	1 line	Lookup table	Formula	Platform

Key Takeaway

Which method to use:

- **Closest to Eppo in these simulations:** Pocock is usually the best match among the DIY options.
- **Avoid over-correction:** Bonferroni is often too conservative, reducing sensitivity more than needed.
- **Avoid early over-triggering:** O'Brien–Fleming can produce too many early significant crossings in this setup.
- **Practical recommendation:** if you are choosing one DIY method for this workflow, use Pocock.

Timing insight: the method choice matters most at the beginning of tests, when monitoring is mostly for implementation issues. As sample size grows, interval widths become more similar across methods.

Implementing the Hybrid Approach Without Eppo

Act 2 introduced the hybrid approach: sequential CI on guardrail KPIs for early abort, standard CI on the primary KPI at the planned end date. Below is how to implement it using any of the three correction methods above.

Step 1: Classify your metrics

Category	Examples	Testing method
Primary KPI	Conversion rate, sign-ups	Fixed-horizon ($z = 1.96$)

Category	Examples	Testing method
Guardrail KPIs	Revenue, error rate, latency, crash rate	Sequential (any of the 3 methods)

Step 2: Set guardrail significance levels

If you have G guardrail KPIs, set $\alpha_{\text{guardrail}} = \alpha/G$ for each one.

Example: 3 guardrails ($G = 3$), $\alpha = 0.05$, 4 weekly peeks ($K = 4$):

$$\alpha_{\text{per guardrail}} = \frac{0.05}{3} \approx 0.0167$$

Then apply Bonferroni within each guardrail:

$$z^* = \Phi^{-1}\left(1 - \tfrac{0.0167}{2\times 4}\right) = \Phi^{-1}(0.9979) \approx 2.86$$

Step 3: Run the experiment

At each scheduled peek:

1. For **each guardrail KPI**: compute the CI using your chosen sequential method.
2. If **any** guardrail CI is entirely on the harmful side of zero: **ABORT** the experiment.
3. Otherwise: continue.

At the **end** of the experiment:

1. For the **primary KPI**: compute a standard CI with the full α:

$$\text{CI}_{\text{primary}} = \hat{\tau} \ \pm \ z_{\alpha/2} \cdot \text{SE}$$

SHIP if the CI excludes zero. Otherwise: no significant effect.

Key Takeaway

The hybrid approach without Eppo:

- Apply Bonferroni across your G guardrail KPIs: $\alpha_g = \alpha/G$.
- Within each guardrail, use O'Brien–Fleming (best) or Bonferroni (simplest).
- Test the primary KPI once at the end with a standard CI — no correction

needed.

- **This is the recommended approach for teams without a sequential testing platform.**

Appendix

Show the code of the simulations

All three simulations on this page use the same component: `ABTestSim.tsx`. It is called with different `layers` props (Act 1: fixed-ci only; Act 2: fixed-ci + sequential-ci; Act 3: all five methods). The full TypeScript source is shown below.

```
"use client"

import { useState, useEffect, useMemo, useRef, type ReactNode } from 'react'
import * as d3 from 'd3'
import { InlineMath } from '../ui/Math'

export type SimLayer =
  | 'fixed-ci'
  | 'sequential-ci'
  | 'pocock'
  | 'obf'
  | 'bonferroni'

type KProp = number;
interface ABTestSimProps {
  layers: SimLayer[]
  showPeekStats?: boolean
  takeaway?: ReactNode
  simulationTitle?: string
  defaultEffect?: number // difference in means between B and A
  defaultN?: number
  showPowerControl?: boolean
  K?: KProp // number of peeks for group sequential methods
  hideEffectStats?: boolean // hide sample effect and CI half-width boxes
}

const Z_975 = 1.959964
const PEEK_N_SIMS = 1000 // number of re-randomizations used to estimate crossing probabiliti
const ALPHA_MIN = 0.01
const ALPHA_MAX = 0.1
const ALPHA_DEFAULT = 0.05

// Normal quantile approximation
function erfinv(x: number) {
  const a = 0.147
  const ln = Math.log(1 - x * x)
  const part1 = 2 / (Math.PI * a) + ln / 2
  const part2 = ln / a
  return Math.sign(x) * Math.sqrt(Math.sqrt(part1 * part1 - part2) - part1)
}
```

```
function normInv(p: number) {
  return Math.sqrt(2) * erfinv(2 * p - 1)
}

function normalCDF(x: number) {
  const a1 = 0.254829592
  const a2 = -0.284496736
  const a3 = 1.421413741
  const a4 = -1.453152027
  const a5 = 1.061405429
  const p = 0.3275911
  const sign = x < 0 ? -1 : 1
  const ax = Math.abs(x) / Math.SQRT2
  const t = 1 / (1 + p * ax)
  const y = 1 - (((((a5 * t + a4) * t) + a3) * t + a2) * t + a1) * t * Math.exp(-ax * ax)
  return 0.5 * (1 + sign * y)
}

function getPeekIndices(n: number, k: number): number[] {
  if (n <= 0 || k <= 0) return []
  const uniqueLooks = new Set<number>()
  for (let j = 1; j <= k; j++) {
    const look = Math.round((j * n) / k)
    uniqueLooks.add(Math.max(1, Math.min(n, look)))
  }
  return Array.from(uniqueLooks).sort((a, b) => a - b)
}

function clampProbability(p: number): number {
  return Math.max(1e-6, Math.min(1 - 1e-6, p))
}

const LAYER_STYLE: Record<SimLayer, { color: string; label: string }> = {
  'fixed-ci':        { color: '#ef4444', label: 'Standard 95% CI' },
  'sequential-ci':   { color: '#2563eb', label: 'Sequential CI (Eppo)' },
  'pocock':          { color: '#f59e0b', label: 'Pocock' },
  'obf':             { color: '#8b5cf6', label: "O'Brien-Fleming" },
  'bonferroni':      { color: '#0d9488', label: 'Bonferroni' },
}

function mulberry32(seed: number) {
  return function () {
    let t = (seed += 0x6d2b79f5)
    t = Math.imul(t ^ (t >>> 15), t | 1)
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61)
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296
  }
}

/** Simulate running difference in means for A/B test with relative lift on a control baselin
function simulateABTestTrajectory(n: number, relativeLift: number, controlRate: number, seed:
  const rand = mulberry32(seed)
  const meansA = new Float64Array(n)
  const meansB = new Float64Array(n)
  const ses = new Float64Array(n)
  const pA = clampProbability(controlRate)
  const pB = clampProbability(pA * (1 + relativeLift))
  let sumA = 0, sumB = 0
  for (let i = 0; i < n; i++) {
```

```
    sumA += rand() < pA ? 1 : 0
    sumB += rand() < pB ? 1 : 0
    const k = i + 1
    meansA[i] = sumA / k
    meansB[i] = sumB / k
    // Pooled variance for difference in means
    const vA = Math.max(meansA[i] * (1 - meansA[i]), 1e-4)
    const vB = Math.max(meansB[i] * (1 - meansB[i]), 1e-4)
    ses[i] = Math.sqrt((vA + vB) / k)
  }
  return { meansA, meansB, ses }
}

export function ABTestSim({
  layers,
  showPeekStats = false,
  takeaway,
  simulationTitle = 'Simulation 1: fixed-horizon confidence intervals.',
  defaultEffect = 0,
  defaultN = 10000,
  showPowerControl = true,
  K: KProp = 6,
  hideEffectStats = false,
}: ABTestSimProps) {
  const [effect, setEffect] = useState(defaultEffect)
  const [n, setN] = useState(defaultN)
  const [alpha, setAlpha] = useState(ALPHA_DEFAULT)
  const [baselineRate, setBaselineRate] = useState(0.1)
  const [seed, setSeed] = useState(1)
  const [kState, setK] = useState(KProp)
  const [peekProbs, setPeekProbs] = useState<Record<string, number> | null>(null)
  const [runSimulationsTrigger, setRunSimulationsTrigger] = useState(0)
  const [showSimulationNotes, setShowSimulationNotes] = useState(false)
  const svgRef = useRef<SVGSVGElement | null>(null)

  const clampedEffect = Math.max(-0.5, Math.min(0.5, effect))
  const effectiveEffect = clampedEffect
  const effectPercent = Math.round(effectiveEffect * 100)
  const clampedBaseline = clampProbability(baselineRate)
  const peekIndices = useMemo(() => getPeekIndices(n, kState), [n, kState])

  // Classical power approximation for two-sided difference-in-proportions test.
  const estimatedPower = useMemo<number | null>(() => {
    if (n <= 0) return null
    if (Math.abs(clampedEffect) < 1e-12) return null
    const pA = clampedBaseline
    const pB = clampProbability(pA * (1 + clampedEffect))
    const delta = Math.abs(pB - pA)
    const seAlt = Math.sqrt((pA * (1 - pA) + pB * (1 - pB)) / n)
    if (seAlt <= 0) return null
    const mu = delta / seAlt
    const zCrit = normInv(1 - alpha / 2)
    const powerVal = normalCDF(mu - zCrit) + normalCDF(-mu - zCrit)
    return Math.max(0, Math.min(1, powerVal))
  }, [n, clampedEffect, alpha, clampedBaseline])

  // Compute the probability of crossing the CI at any point for all selected layers
  useEffect(() => {
    if (!showPeekStats || runSimulationsTrigger === 0) return;
    const results: Record<string, number> = {};
```

```
    for (const layer of layers) {
      let count = 0;
      for (let sim = 0; sim < PEEK_N_SIMS; ++sim) {
        const t = simulateABTestTrajectory(n, effectiveEffect, clampedBaseline, seed + sim +
        let crossed = false;
        if (peekIndices.length === 0) continue;
        let lookPtr = 0
        const lastLookPtr = peekIndices.length - 1
        for (let i = 0; i < n && lookPtr <= lastLookPtr; ++i) {
          if ((i + 1) !== peekIndices[lookPtr]) continue;
          const denom = t.meansA[i];
          const est = denom !== 0 ? 100 * (t.meansB[i] - denom) / denom : 0;
          let w = 0;
          if (layer === 'fixed-ci') {
            w = denom !== 0 ? 100 * Z_975 * t.ses[i] / denom : 0;
          } else if (layer === 'sequential-ci') {
            const nu = n * 0.25;
            const t_i = i + 1;
            const logTerm = Math.log((t_i + nu) / (nu * alpha));
            w = denom !== 0 ? 100 * t.ses[i] * Math.sqrt((t_i + nu) / t_i * logTerm) / denom
          } else if (layer === 'pocock') {
            const cP = 2.41;
            w = denom !== 0 ? 100 * t.ses[i] * cP / denom : 0;
          } else if (layer === 'obf') {
            const k = Math.max(1, Math.round((i + 1) / n * kState));
            const z = normInv(1 - alpha / (2 * kState / k));
            w = denom !== 0 ? 100 * t.ses[i] * z / denom : 0;
          } else if (layer === 'bonferroni') {
            const z = normInv(1 - alpha / (2 * kState));
            w = denom !== 0 ? 100 * t.ses[i] * z / denom : 0;
          }
          if (est - w > 0 || est + w < 0) {
            crossed = true;
            break
          }
          lookPtr++
        }
        if (crossed) count++;
      }
      results[layer] = count / PEEK_N_SIMS;
    }
    setPeekProbs(results);
  }, [showPeekStats, layers, n, clampedEffect, effectiveEffect, clampedBaseline, seed, alpha,

  const traj = useMemo(
    () => simulateABTestTrajectory(n, effectiveEffect, clampedBaseline, seed),
    [n, effectiveEffect, clampedBaseline, seed]
  )

  // Compute the running effect in percent: (meanB - meanA) / meanA
  const effectPct = useMemo(() => {
    const arr = new Float64Array(n)
    for (let i = 0; i < n; i++) {
      const denom = traj.meansA[i]
      arr[i] = denom !== 0 ? 100 * (traj.meansB[i] - denom) / denom : 0
    }
    if (n > 0) arr[0] = 0
    return arr
  }, [traj, n])
```

```
// Compute the per-step CI half-width for the fixed CI (in percent)
const ciHalfWidthPct = useMemo(() => {
  const arr = new Float64Array(n)
  for (let i = 0; i < n; i++) {
    const denom = traj.meansA[i]
    arr[i] = denom !== 0 ? 100 * Z_975 * traj.ses[i] / denom : 0
  }
  if (n > 0) arr[0] = 0
  return arr
}, [traj, n])

// Render the plot with D3
useEffect(() => {
  if (!svgRef.current) return
  const svg = d3.select(svgRef.current)
  svg.selectAll('*').remove()
  const W = 700, H = 360
  const margin = { top: 20, right: 16, bottom: 44, left: 56 }
  const innerW = W - margin.left - margin.right
  const innerH = H - margin.top - margin.bottom
  const yMin = -100, yMax = 100
  const xUpper = Math.max(1, n)
  const x = d3.scaleLinear().domain([0, xUpper]).range([0, innerW])
  const y = d3.scaleLinear().domain([yMin, yMax]).range([innerH, 0])
  const g = svg.append('g').attr('transform', `translate(${margin.left},${margin.top})`)
  const xAxisY = y(0)

  g.append('g').attr('transform', `translate(0,${xAxisY})`).call(d3.axisBottom(x).ticks(6))
  g.append('g').call(d3.axisLeft(y).ticks(6).tickFormat(d => `${Math.round(d as number)}%`)
  g.append('text')
    .attr('transform', `translate(${innerW / 2}, ${innerH + 34})`)
    .style('text-anchor', 'middle').style('font-size', '12px').style('fill', '#525252')
    .text('Number of users (n)')
  g.append('text')
    .attr('transform', 'rotate(-90)')
    .attr('x', -innerH / 2).attr('y', -42)
    .style('text-anchor', 'middle').style('font-size', '12px').style('fill', '#525252')
    .text('Effect (%)')

  // Reference line at 0
  g.append('line')
    .attr('x1', 0).attr('x2', innerW)
    .attr('y1', y(0)).attr('y2', y(0))
    .attr('stroke', '#525252').attr('stroke-width', 1).attr('stroke-dasharray', '4 3')

  // CI bands — one per layer
  const ciWidth = (layer: SimLayer, i: number) => {
    const denom = traj.meansA[i]
    if (denom === 0) return 0
    if (layer === 'fixed-ci') return 100 * Z_975 * traj.ses[i] / denom
    if (layer === 'sequential-ci') {
      const nu = n * 0.25
      const t_i = i + 1
      return 100 * traj.ses[i] * Math.sqrt((t_i + nu) / t_i * Math.log((t_i + nu) / (nu * a
    }
    if (layer === 'pocock') return 100 * traj.ses[i] * 2.41 / denom
    if (layer === 'obf') {
      const k = Math.max(1, Math.round((i + 1) / n * kState))
      return 100 * traj.ses[i] * normInv(1 - alpha / (2 * kState / k)) / denom
    }
```

```
      if (layer === 'bonferroni') return 100 * traj.ses[i] * normInv(1 - alpha / (2 * kState)
      return 0
    }

    for (const layer of layers) {
      const area = d3.area<number>()
        .x((_d, i) => x(i + 1))
        .y0((_d, i) => y(effectPct[i] - ciWidth(layer, i)))
        .y1((_d, i) => y(effectPct[i] + ciWidth(layer, i)))
        .defined((_d, i) => i >= 5 && Number.isFinite(ciWidth(layer, i)))
      g.append('path')
        .datum(Array.from({ length: n }, (_, i) => i))
        .attr('fill', LAYER_STYLE[layer].color)
        .attr('fill-opacity', 0.12)
        .attr('stroke', LAYER_STYLE[layer].color)
        .attr('stroke-width', 1.2)
        .attr('stroke-opacity', 0.8)
        .attr('d', area as d3.Area<number>)
    }

    // Mean effect trajectory
    const line = d3.line<number>()
      .x((_d, i) => x(i + 1))
      .y((_d, i) => y(effectPct[i]))
    g.append('path')
      .datum(Array.from({ length: n }, (_, i) => i))
      .attr('fill', 'none').attr('stroke', '#0f172a').attr('stroke-width', 1.6)
      .attr('d', line as d3.Line<number>)
  }, [effectPct, ciHalfWidthPct, n, layers, traj, alpha, kState])

  const decision = useMemo(() => {
    if (n <= 0 || peekIndices.length === 0) return null
    for (const look of peekIndices) {
      const i = look - 1
      const lo = effectPct[i] - ciHalfWidthPct[i]
      const hi = effectPct[i] + ciHalfWidthPct[i]
      if (lo > 0 || hi < 0) return { label: 'Yes' }
    }
    return { label: 'No' }
  }, [effectPct, ciHalfWidthPct, n, peekIndices])

  return (
    <div className="bg-white border border-neutral-300 rounded-lg p-4 my-6">
      {/* Controls: baseline rate, n, alpha, effect size, K, estimated power */}
      {/* ... (see full source for complete JSX) */}

      {/* SVG plot rendered by D3 */}
      <svg ref={svgRef} viewBox="0 0 700 360" style={{ minWidth: 700, width: '100%' }} />

      {/* Decision panel, peek-probability runner, simulation notes */}
    </div>
  )
}
```

Summary

Act	Key message
1	Peeking inflates false positives. You need a test that is valid at every time you check.
2	Sequential confidence intervals (CIs) are valid at every moment. The hybrid approach uses sequential CIs for guardrails and classical CIs for the primary KPI.
3	DIY alternatives: Bonferroni, Pocock, O'Brien–Fleming. All widen the CI to control error when peeking.
4–6	Martingales and Ville's inequality: the math that makes peeking safe.
7–9	SPRT, mixture, and confidence sequences: the evolution of sequential testing and estimation.
10–13	Eppo's pipeline: regression adjustment, sequential CIs, and actionable decisions.

The core trade-off: Valid interim monitoring requires wider confidence intervals. The penalty depends on the method, but the benefit is freedom to peek and act at any time.

Method	Price of peeking
No correction (naive)	Invalid inference (20–30% false positives)
Bonferroni	CI ~28–68% wider (depending on K)
Pocock	CI ~20–37% wider
O'Brien–Fleming	CI ~3% wider at final analysis
Eppo (sequential CI)	CI ~10–40% wider, valid *continuously*
Hybrid (recommended)	No penalty on primary KPI; sequential penalty on guardrails only

Mathematical Reference

Notation

Symbol	Meaning	Where used
Y_i	Outcome for user i during the experiment	Act 2, Steps 2–3
X_i	Pre-experiment covariate for user i	Act 2, Step 3 (CUPED)
Y_i^*	Adjusted outcome after regression adjustment	Act 2, Step 3
$\hat{\tau}(t)$	Estimated treatment effect at time t	Act 2, Step 4
$\hat{\mu}_0(t),\ \hat{\mu}_1(t)$	Adjusted group means at time t	Act 2, Step 4
$\hat{\sigma}_{\hat{\tau}}(t)$	Estimated standard error of treatment effect	Act 2, Step 5
$s_g^2(t)$	Sample variance in group g at time t	Act 2, Step 5
$n_0(t),\ n_1(t)$	Number of users in control / treatment at time t	Throughout
n	Total observations: n_0(t) + n_1(t)	Act 2, Step 6
α	Significance level (typically 0.05)	Throughout
ν	Tuning parameter: \nu = M \cdot \hat{\sigma}^2	Act 2, Step 6 (sequential CI)
M	Expected total sample size	Act 2, Step 6
K	Number of pre-planned analysis times (peeks)	Act 3
k	Current peek number (1 \leq k \leq K)	Act 3

Symbol	Meaning	Where used
t_k	Information fraction at peek k: t_k = k/K	Act 3 (OBF)
Z_k	z-statistic at peek k: \hat{\tau}/\text{SE}	Act 3
$z_{\alpha/2}$	Standard Normal critical value (\approx 1.96)	Throughout
$z_{\alpha/(2K)}$	Bonferroni-adjusted critical value	Act 3 (Bonferroni)
c_P	Pocock constant critical value	Act 3 (Pocock)
c_k^{OBF}	O'Brien–Fleming critical value at peek k	Act 3 (OBF)
Φ^{-1}	Inverse standard Normal CDF (quantile function)	Act 3
ρ	Correlation between pre-experiment and experiment metrics	Act 2, Step 3
G	Number of guardrail KPIs	Act 3 (Hybrid)

Key Formulas

Standard Confidence Interval

$$\text{CI} = \hat{\tau} \ \pm \ z_{\alpha/2} \cdot \text{SE}$$

Valid only at one pre-specified time. The constant $z_{\alpha/2} \approx 1.96$ for a 95% CI.

Sequential Confidence Interval (Howard et al., 2021)

$$\text{CI}(t) = \hat{\tau}(t) \ \pm \ \hat{\sigma}_{\hat{\tau}}(t) \cdot \sqrt{\frac{n+\nu}{n} \cdot \log \frac{n+\nu}{\nu\alpha^2}}$$

Valid at *all times simultaneously*. The multiplier starts high (~3.8) and decreases toward 1.96.

CUPED Variance Reduction

$$Y_i^* = Y_i - \hat{f}_g(X_i) \qquad \mathrm{Var}(Y^*) = \mathrm{Var}(Y)(1-\rho^2)$$

At $\rho = 0.7$, half the variance is eliminated.

Treatment Effect & Standard Error

$$\hat{\tau} = \bar{Y}_1 - \bar{Y}_0 \qquad \mathrm{SE} = \sqrt{\frac{s_0^2}{n_0} + \frac{s_1^2}{n_1}}$$

Bonferroni Adjusted CI

$$\mathrm{CI}_k = \hat{\tau}(t_k) \ \pm \ z_{\alpha/(2K)} \cdot \mathrm{SE}(t_k)$$

O'Brien–Fleming Boundary

$$c_k^{\mathrm{OBF}} = \frac{z_{\alpha/2}}{\sqrt{t_k}} = z_{\alpha/2} \cdot \sqrt{\frac{K}{k}}$$

Pocock Boundary

$$\mathbb{P}\Big(\max_{1\le k\le K} |Z_k| > c_P\Big) = \alpha$$

With correlation: $\mathrm{Cor}(Z_j, Z_k) = \sqrt{\min(j,k)/\max(j,k)}$

Detailed Explanations

Confidence Sequences

A confidence sequence is a sequence of confidence sets $(C_t)_{t\geq 1}$ such that the true parameter lies in C_t for *all* t simultaneously with high probability. Unlike a standard CI (valid at one time), a confidence sequence allows continuous monitoring. The price: wider intervals, especially early on.

Alpha Spending

The total false positive rate αcan be thought of as a “budget” that is spent across analyses. Bonferroni spends α/Kat each peek. O'Brien–Fleming spends almost nothing early and most at the end. The alpha-spending function $\alpha^*(t)$ formalises this: at information fraction t, cumulative type I error must not exceed $\alpha^*(t)$.

Information Fraction

The information fraction at peek k is $t_k = k/K$ for equally-spaced analyses, or more generally the ratio of current to planned total statistical information (proportional to sample size for equal variances). It ranges from 0 to 1 and determines where you are in the experiment.

FWER vs FDR

FWER (Family-Wise Error Rate): the probability of making *at least one* false positive across all tests. Bonferroni, Pocock, and OBF all control FWER. **FDR** (False Discovery Rate): the expected *proportion* of false positives among rejected hypotheses. FDR control is less strict and typically used when testing many hypotheses simultaneously (e.g. gene expression). For A/B testing with a small number of metrics, FWER control is standard.

References

1. **Deng, A., Xu, Y., Kohavi, R., & Walker, T.** (2013). Improving the sensitivity of online controlled experiments by utilizing pre-experiment data. *Proc. WSDM*, 123–132. — The original CUPED paper. Variance reduction via pre-experiment covariates.

2. **Howard, S.R., Ramdas, A., McAuliffe, J., & Sekhon, J.** (2021). Time-uniform, nonparametric, nonasymptotic confidence sequences. *Annals of Statistics*, 49(2), 1055–1080. — The confidence sequence framework that Eppo adopted.

3. **Pocock, S.J.** (1977). Group sequential methods in the design and analysis of clinical trials. *Biometrika*, 64(2), 191–199.

4. **O'Brien, P.C. & Fleming, T.R.** (1979). A multiple testing procedure for clinical trials. *Biometrics*, 35(3), 549–556.

5. **Schmit, M. & Miller, T.** (2024). Sequential confidence intervals for comparing two proportions with variance reduction. *Eppo Technical Report*. — Eppo's implementation combining Howard et al.'s confidence sequences with generalised CUPED.

Comments

Sign in with GitHub to leave a comment. When commenting for the first time, please add your title and institution / workplace at the top of your message. Comments are moderated and appear after approval.

0 reactions

0 comments

Write Preview Aa

Sign in to comment

Sign in with GitHub

Contact us

Found an error? Have a suggestion for improvement? Please let us know!

Name

Email

Message

Send message